models *15W* performed best. For *Q4_0* quantization, InternLM 2.5 7B was the first model, for which *15W* performed better, contrarily to *Q4_K_M* quantization wer this transition was already observable for Yi 1.5 6B. However, 2 outliers can be identified for *Q4_K_M*quantization. For Qwen 2 0.5B despite being the smallest model, the *15W* mode and for InterLM 2.5 7B despite being one of the larger models, the *MAX* mode performed best.

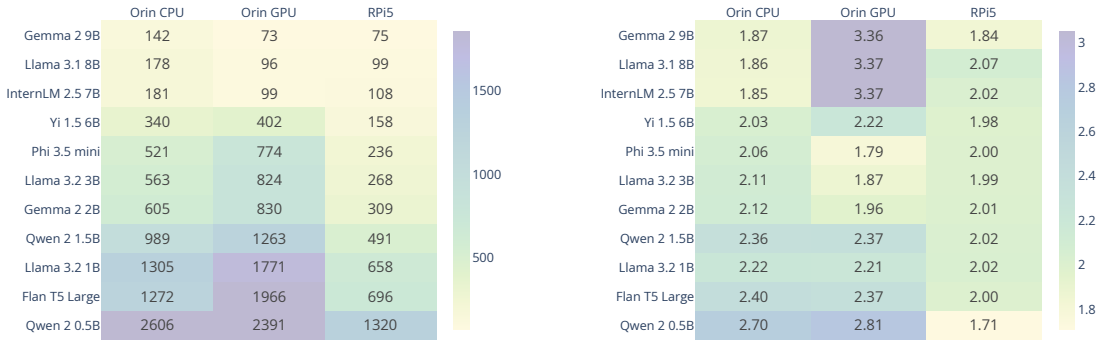
In the generation phase, the *MAX* mode yielded the optimal trade-off for all models up to Llama 3.2 3B with *Q4_K_M* quantization and up to Phi 3.5 with *Q4_0* quantization. However, for larger models, the *15W* mode proved to be more efficient. Results for the *7W* mode were omitted, as it performed worse than the other two power modes in both token throughput and energy consumption.

Overall, a clear trend emerged: the *MAX* mode provided better performance for smaller models, whereas the *15W* mode was more efficient for larger models.

Main Takeaways

- For both prefill and generation phase on the RPi 5, the *ondemand* and *performance* CPU governors performed best with both quantization schemes. In the generation phase, 2 threads was the optimal thread count for most models.
- For GPU inference, the *MAX* mode showed the optimal throughput-energy trade-off for most smaller models while for larger models the *15W* mode showcased the best throughput-energy efficiency.

6.2 Usability Constraints and Practical Deployment Challenges



(a) Runs (b) Time [hours]

Fig. 16. Number of inference runs and Time [hours] until battery recharge for configurations with minimal energy consumption per run

In the following, we analyze how inference speed and energy consumption impact the usability of LMs in real-world systems.

One key challenge is the long load times observed during the first run of each experiment and for all runs of larger models on the Orin. In any interactive use case, where users expect real-time responses, these high loading times—which translate to significant latencies—would likely degrade the user experience. To mitigate this, it is crucial to keep the model cached in memory between runs, as this substantially reduces

load times, as discussed in Section 5. It can also be considered to pre-load models in idle times even before a request arrives to hide the latency from the user.

Another potential limitation is the high memory demand of the models, even when they fit within the device's main memory. First, our results indicate that for the largest evaluated models, complications arose during GPU inference across both quantization schemes, as well as in CPU inference with *Q4_0* quantization, leading to significantly increased loading times and reduced prefill performance. Second, in real-world deployments, edge devices are often not dedicated solely to LM inference but may need to support additional applications. If the inference process occupies the majority of system memory, other applications may fail to run concurrently, leading to frequent memory paging. Since storage accesses are inherently time-consuming, this could cause substantial performance degradation.

For use cases where human users read generated tokens in real-time, reading speed serves as a baseline to determine whether token throughput is sufficient to maintain a smooth user experience. If tokens are generated slower than the average reading speed, users may need to pause frequently, leading to disruptions and a degraded experience [28]. The average reading speed of an English reader is approximately 4 words per second [7]. According to OpenAI, one token corresponds to roughly 0.75 words [44]. Based on this, an optimal generation throughput should be at least 5.3 $\frac{tokens}{s}$ to ensure a seamless experience. However, human reading speed varies significantly between individuals; therefore, this threshold should be considered only as an approximate guideline for assessing generation speed requirements in such use cases. When selecting the highest generation throughput for each model on the RPi 5, only models up to Llama 3.2 3B with *Q4_K_M* quantization and up to Phi 3.5 mini with *Q4_0* quantization were able to exceed the 5.3 $\frac{tokens}{s}$ threshold. For Orin CPU inference, all models up to Yi 1.5 6B with *Q4_K_M* quantization and up to InternLM 2.5 7B with *Q4_0* quantization met this requirement. However, for larger models, a degraded user experience seems likely due to insufficient generation throughput. In contrast, on the Orin with GPU inference, all models exceeded 5.3 $\frac{tokens}{s}$ for both quantization schemes, ensuring a smoother reading experience.

However, generation speed requirements depend on the specific use case. In some scenarios, lower generation speeds may be sufficient, while in others, higher speeds could be advantageous. For instance, if an LM is summarizing documents as a background process—without user interaction or strict time constraints—slower generation speeds may be acceptable. In contrast, multi-agent LM systems could benefit from higher generation speeds, as LMs can typically process input tokens much faster than human reading speed.

As edge devices are often battery-powered or have limited energy availability, it is important to assess how many inference runs can be performed before the device runs out of battery. For each model, we used the configuration with the lowest mean energy consumption per run (considering runs 2–5 of an experiment) to estimate the number of inference runs that could be executed at 100% utilization (i.e., with no idle time) on a fully charged smartphone battery. As a representative example, we selected the Samsung Galaxy S24 Ultra, a high-end smartphone with a nominal battery capacity of $18.84Wh = 67824J$. As it is observable, even the maximum runtime remained below 3.5 hours, raising concerns about the feasibility of LM inference in energy-constrained environments, particularly when inference is performed frequently. That said, it is important to acknowledge the caveats of this analysis. First, our analysis considers only one input prompt, whereas different prompts could lead to variations in both the number of inference runs and total runtime before recharging. Second, the configurations minimizing total energy consumption were used for

this evaluation. On the RPi 5, this often corresponds to the *powersave* governor, which reduces inference speed. In real-world applications, this performance might be too slow, necessitating a less energy-efficient configuration. Because of this, these results should be interpreted as an upper bound on the number of feasible inference runs.

Main Takeaways

- It is doubtful whether generation speed of edge in inference is satisfying for the largest evaluated models for a wide range of use cases, especially with CPU inference
- However, performance for smaller models is likely satisfactory for many uses cases as generation throughput exceeded average human read speed even with CPU inference
- Due to the high energy consumption, it is questionable whether highly frequent LM inference is feasible in energy-constrained environment

6.3 Cost Analysis

With self-deployed models at the edge, no service costs are incurred for the user in contrast to the API costs of cloud services. For instance, at the time of writing, executing inference with the *GPT-4o mini* model via the OpenAI API incurs a cost of 15¢ per million input tokens and 60¢ per million output tokens[1] [43].

Model	Input			Output		
	Orin CPU	Orin GPU	RPi5	Orin CPU	Orin GPU	RPi5
G2-9B	3.369	0.759	6.282	14.856	10.227	32.906
L3.1-8B	3.126	0.711	5.211	10.983	7.752	25.432
I2.5-7B	2.924	0.656	4.570	10.485	7.529	24.605
Y1.5-6B	2.277	0.353	4.241	8.299	6.160	19.306
P3.5-M	1.365	0.165	2.447	5.414	4.225	12.875
L3.2-3B	1.141	0.115	2.064	5.128	3.777	11.595
G2-2B	0.842	0.103	1.486	5.388	3.829	10.637
Q2-1.5B	0.485	0.092	0.956	2.705	2.119	5.770
L3.2-1B	0.347	0.061	0.715	2.265	1.629	4.790
FT5-L	0.152	0.074	0.272	2.822	1.684	5.321
Q2-0.5B	0.126	0.065	0.238	1.014	1.037	2.057

Fig. 17. Energy cost per one million input and output tokens for different devices in cents

However, edge deployments also incur operational costs, as energy consumption contributes to long-term expenses, in addition to the initial device purchase price. Using energy measurements from our experiments, we computed the cost per one million input and output tokens (illustrated in Figure 17). The calculations assume an optimal thread count with the *ondemand* governor for CPU inference, while for GPU inference, the *15W* power mode was selected as the most energy-efficient setting. An electricity price of 20¢/kWh was chosen as a middle ground, considering that energy costs vary significantly by location and sector. For example, in the U.S., electricity prices in September 2024 ranged from under 10¢/kWh to over 40¢/kWh [2].

[1] "Ignoring Batch API prices and cached token prices"